

Patrones de Desarrollo Web y MVC



Objetivos de Aprendizaje

- Comprender qué son los patrones de desarrollo de software
- Conocer los patrones más comunes en desarrollo de software
- Entender el patrón MVC
- Aplicar MVC en la estructuración de proyectos web
- Crear diagramas de arquitectura usando MVC



Introducción a los Patrones de Desarrollo



Fundamentos de Patrones de Desarrollo

Definición: Los patrones de desarrollo son soluciones reutilizables a problemas comunes en el diseño de software.

Analogía: Como los planos arquitectónicos para construir casa, los patrones son "planos" para organizar código

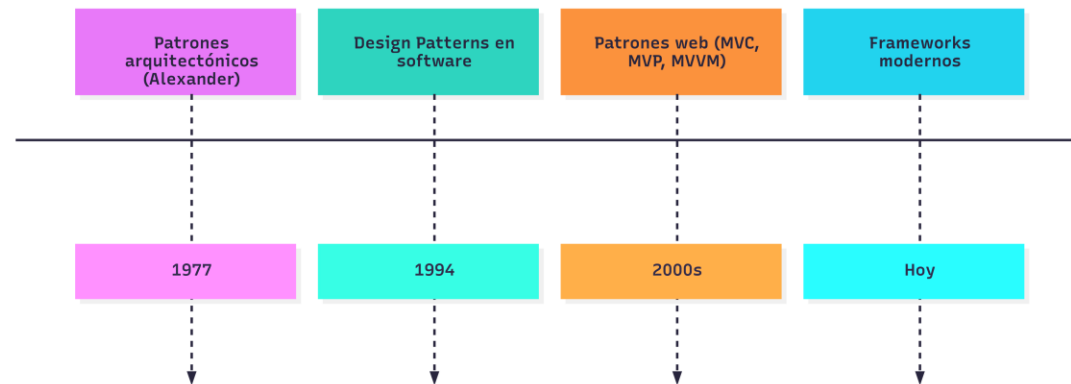
Beneficios principales

- Código más organizado y mantenible
- Comunicación clara entre desarrolladores
- Soluciones probadas y eficientes
- Escalabilidad del proyecto



De Dónde Vienen los Patrones

- **Origen:** Inspirados en los patrones de diseño arquitectónico de Christopher Alexander
- **En software:** Popularizados por el libro "Design Patterns" (Gang of Four, 1994)
- **En desarrollo web:** Adaptados para resolver problemas específicos de aplicaciones web



¿Por Qué Necesitamos Patrones?

- Problemas comunes sin patrones:
 - **Código espagueti:** Todo mezclado, difícil de entender
 - **Dificultad para mantener:** Cambios pequeños requieren tocar muchos archivos
 - **Falta de reutilización:** Código repetido en múltiples lugares
 - **Equipos desorganizados:** Cada desarrollador estructura el código diferente
- **Soluciones con patrones:**
 - Separación clara de responsabilidades
 - Estructura predecible y consistente
 - Facilidad para testing y debugging
 - Colaboración eficiente en equipo



Patrones Comunes en Desarrollo Web



Principales Patrones en Desarrollo Web

- **Patrones de Arquitectura**
 - **MVC (Model-View-Controller)** - Separación en tres capas
 - **MVP (Model-View-Presenter)** - Variante de MVC
 - **MVVM (Model-View-ViewModel)** - Para interfaces complejas
- **Patrones de Diseño:**
 - **Observer** – Para notificaciones automáticas
 - **Factory** – Para crear objetos dinámicamente
 - **Singleton** – Para recursos únicos



Comparativa de Patrones Arquitectónicos

MVC (Model-View-Controller)	MVP (Model-View-Presenter)	MVVM (Model-View-ViewModel)
Controlador maneja la lógica de entrada	Presenter actúa como intermediario	ViewModel une Vista y Modelo
Vista muestra datos al usuario	Vista es más "tonta" (menos lógica en la vista)	Data binding automático
Modelo contiene datos y lógica de negocio	Mejor para testing	Popular en frameworks modernos (más declarativo)



Cuándo Usar Cada Patrón

Usa MVC cuando	Usa MVP cuando	Usa MVVM cuando
Aplicaciones web tradicionales	Necesitas testing extensivo	Interfaces muy dinámicas
Separación clara de responsabilidades	La vista debe ser muy simple	Data binding en tiempo real
Equipos con roles definidos (frontend/backend)	Lógica de presentación compleja	Frameworks como Angular, Vue
Aplicaciones medianas a grandes		



Modelo-Vista-Controlador: Conceptos Básicos

Los Tres Pilares

1. **Modelo (Model):** Los datos y la lógica de negocio
2. **Vista (View):** La interfaz de usuario
3. **Controlador (Controller):** Intermediario que maneja las interacciones

Analogía

- **Modelo:** La cocina (prepara la comida/datos)
- **Vista:** El comedor (dónde se sirve/muestra)
- **Controlador:** El camarero (toma pedidos/maneja interacciones)



El Modelo (Model): Corazón de los Datos

- **Responsabilidades del Modelo:**

- Gestionar los datos de la aplicación
- Implementar la lógica de negocio
- Validar la información
- Comunicar cambios a otros componentes

- **Qué NO debe hacer el Modelo:**

- Saber cómo se muestran los datos
- Manejar eventos de usuario
- Contener lógica de presentación



La Vista (View): Interfaz con el Usuario

- **Responsabilidades de la Vista**

- Mostrar datos al usuario
- Recoger input del usuario
- Renderizar la interfaz
- Presentar información de forma clara
- Responder a eventos de interacción

- **Qué NO debe hacer la Vista**

- Procesar datos complejos
- Conectar directamente con la base de datos
- Contener lógica de negocio
- Tomar decisiones sobre qué mostrar

- **Tecnologías comunes para vistas:**

- HTML + CSS + JavaScript
- Templates (EJS, Handlebars, Pug)
- Frameworks frontend (React, Angular, Vue)



El Controlador (Controller): Director de Orquesta

- **Responsabilidades del Controlador**

- Recibir peticiones del usuario
- Coordinar entre Modelo y Vista
- Decidir qué datos necesita la Vista
- Manejar el flujo de la aplicación
- Gestionar la navegación entre páginas

- **Qué NO debe hacer el Controlador**

- Contener lógica de negocio compleja
- Manejar datos directamente
- Generar HTML o interfaces

- **Flujo típico del Controlador**

- Usuario hace click -> Controlador recibe evento
- Controlador pide datos -> Modelo procesa
- Modelo devuelve datos -> Controlador los recibe
- Controlador envía datos -> Vista los muestra

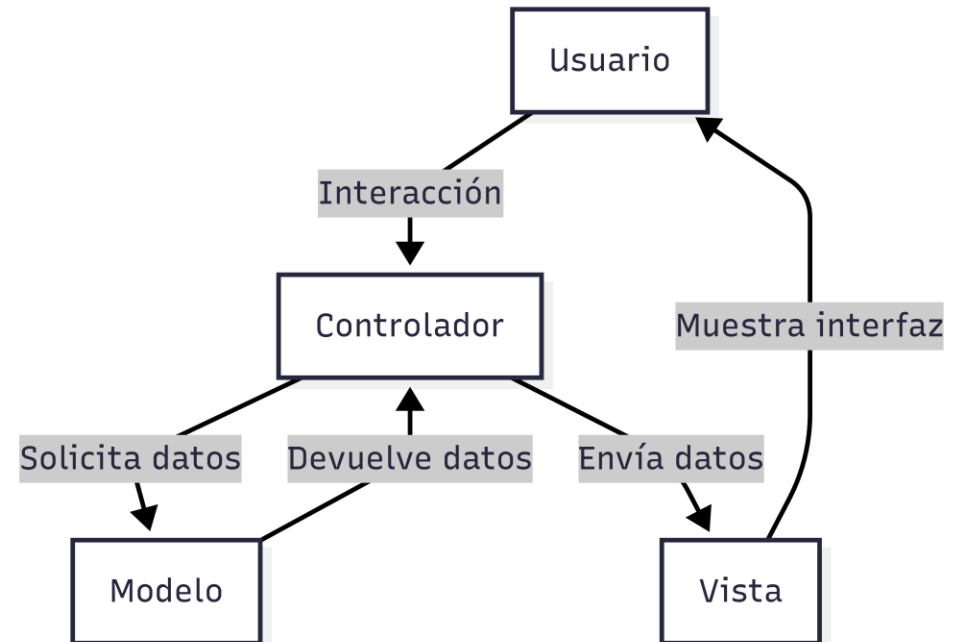


Cómo se Comunican los Componentes

- **Usuario -> Controlador:** Siempre a través del controlador
- **Controlador -> Modelo:** Para obtener/modificar datos
- **Modelo -> Controlador:** Devuelve datos procesados
- **Controlador -> Vista:** Envía datos para mostrar
- **Vista -> Usuario:** Presenta la información final

Prohibiciones importantes:

- Vista NO puede hablar directamente con Modelo
- Modelo NO conoce la existencia de la Vista
- Usuario NO accede directamente al Modelo



Pros y Contras del Patrón MVC

✓ Ventajas

- **Separación clara:** Cada componente tiene un propósito específico
- **Mantenibilidad:** Cambios en una parte no afectan a las otras
- **Reutilización:** Modelos pueden usarse con múltiples vistas
- **Testing:** Fácil probar cada componente por separado
- **Trabajo en equipo:** Diferentes personas pueden trabajar en cada capa
- **Escalabilidad:** Fácil añadir nuevas funcionalidades

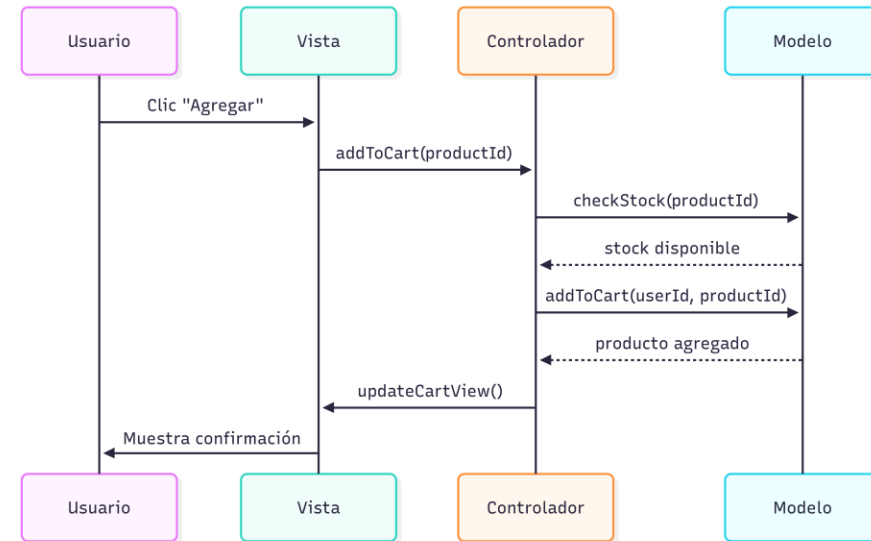
✗ Desventajas

- **Complejidad inicial:** Más archivos y estructura para proyectos simples
- **Curva de aprendizaje:** Requiere entender el patrón primero
- **Sobre-ingeniería:** Puede ser excesivo para aplicaciones muy pequeñas
- **Comunicación overhead:** Más pasos para operaciones simples



¡Ejemplo práctico!

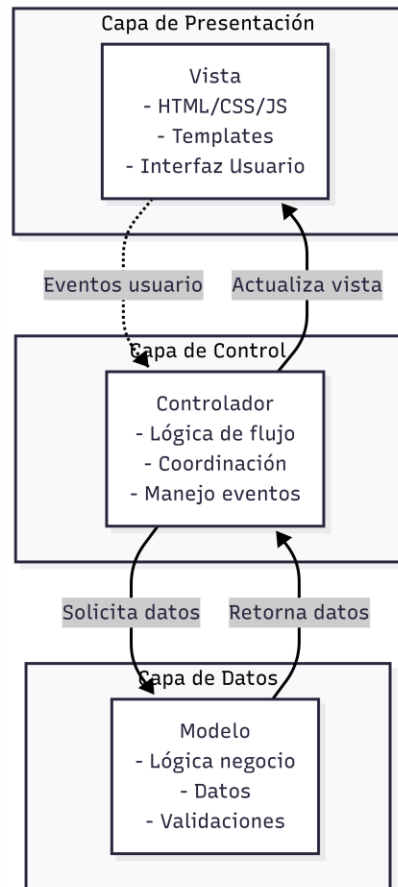
- Paso a paso:
 - **Vista** -> Usuario hace click en botón "Agregar al carrito"
 - **Controlador** -> `CartController.addProduct()` recibe la petición
 - **Controlador** -> Valida que el usuario esté logueado
 - **Modelo** -> `Product.checkStock()` verifica disponibilidad
 - **Modelo** -> `Car.addItem()` añade producto al carrito
 - **Modelo** -> `Product.updateStock()` actualiza inventario
 - **Controlador** -> Recibe confirmación del modelo
 - **Vista** -> Actualiza interfaz mostrando producto agregado



Diagramas de Arquitectura MVC



Visualizando la Arquitectura MVC

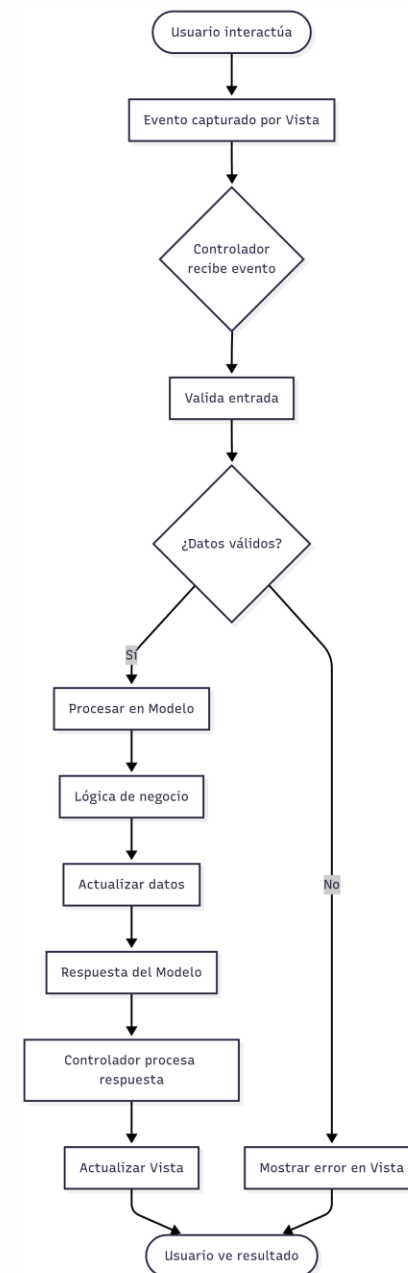


- **Elementos clave:**

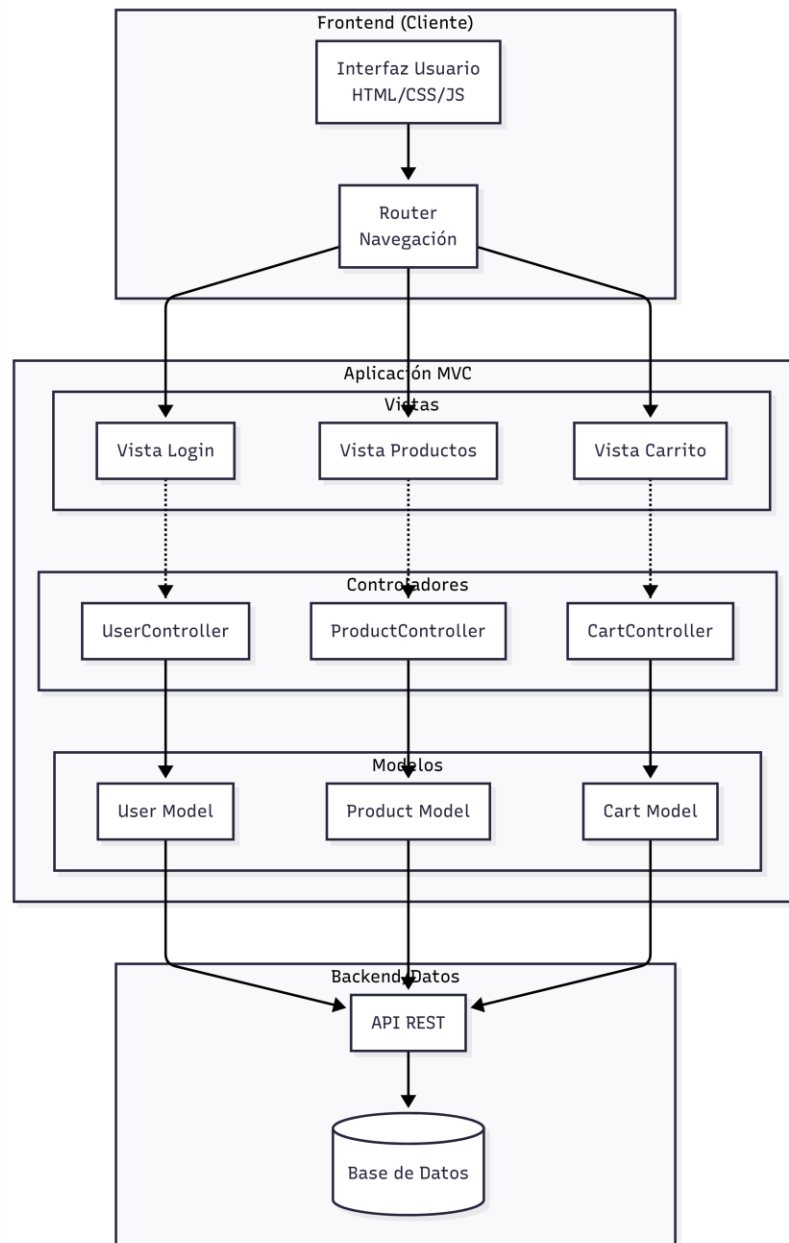
- Separación clara en capas
- Flujo de comunicación unidireccional
- Responsabilidades específicas por capa

Flujo de Datos en Aplicación MVC

- **Elementos clave:**
 - Validación en múltiples niveles
 - Separación clara de responsabilidades
 - Manejo de errores integrado



Ejemplo completo



Buenas Prácticas para Implementar MVC



Haz esto

- Mantén controladores "delgados" (poca lógica)
- Modelos "gordos" (toda la lógica de negocio)
- Vistas "tontas" (solo la presentación)
- Un controlador por funcionalidad principal
- Nombres descriptivos y consistentes
- Testing individual de cada componente



Evita esto

- Lógica de negocio en controladores
- Acceso directo de Vista a Modelo
- Controladores que hacen demasiado
- Modelos que conocen la interfaz
- Mezclar responsabilidades entre capas



LA REGLA DE ORO

"Cada componente debe tener **una**
sola responsabilidad y **hacerla bien.**"

Estrategias de Testing para MVC

Testing de Modelos:

- Unit tests para lógica de negocio
- Validación de datos
- Operaciones CRUD

• Testing de controladores:

- Integration tests
- Verificar flujo de datos
- Manejo de errores

• Testing de Vistas:

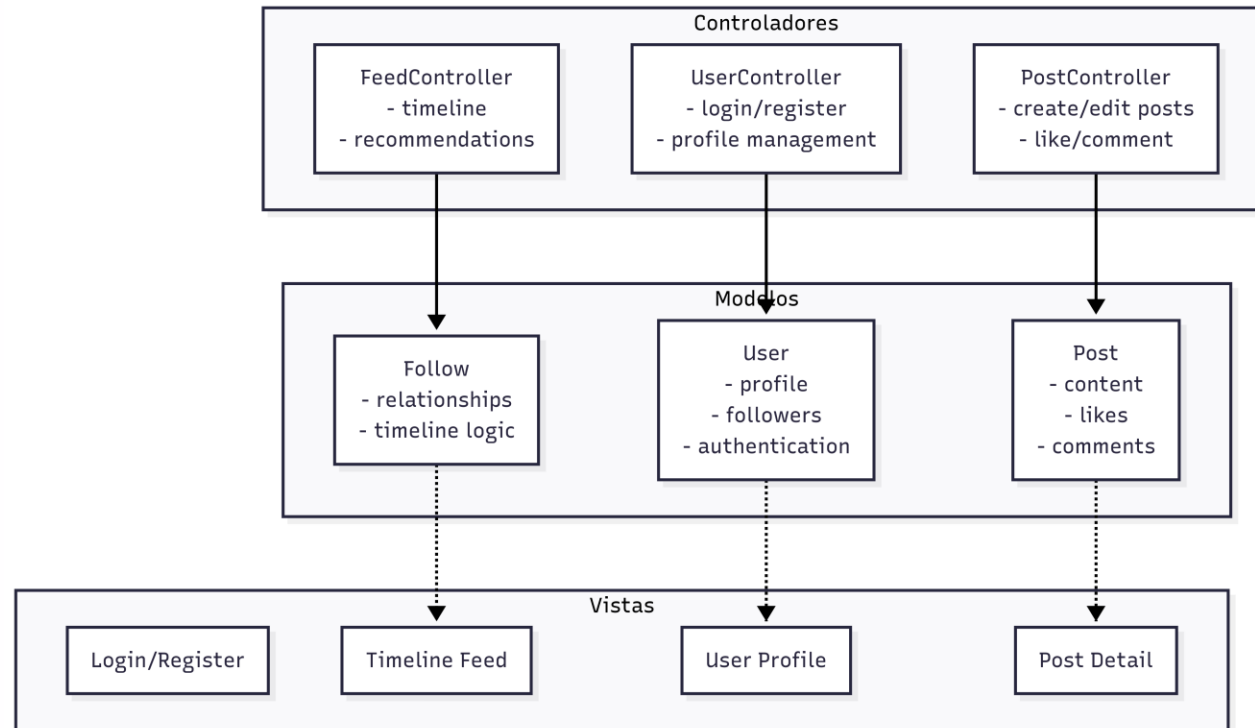
- UI testing
- Interacciones usuario
- Renderizado correcto

• Testing de sistema completo:

- End-to-end tests (E2E)
- Flujos completos de usuario
- Performance testing



Ejemplo de Red Social



Evolución y Futuro de MVC

- **Histórica:**

- 1970s -> MVC en Smalltalk
- 2000s -> MVC en frameworks web
- 2010s -> Variaciones (MVP, MVVM)
- 2020s -> Micro-frontends, JAMstack

- **Tendencias actuales:**

- Micro-frontends: MVC a nivel de componentes
- JAMstack: Separación de frontend estático y APIs
- Serverless: Controladores como funciones
- Real-time: Modelos reactivos con WebSockets

- **¡MVC sigue siendo relevante!**

- Principios fundamentales no cambian
- Se adapta a nuevas tecnologías
- Base para entender patrones modernos
- Escalabilidad probada en aplicaciones grandes



kingscorner